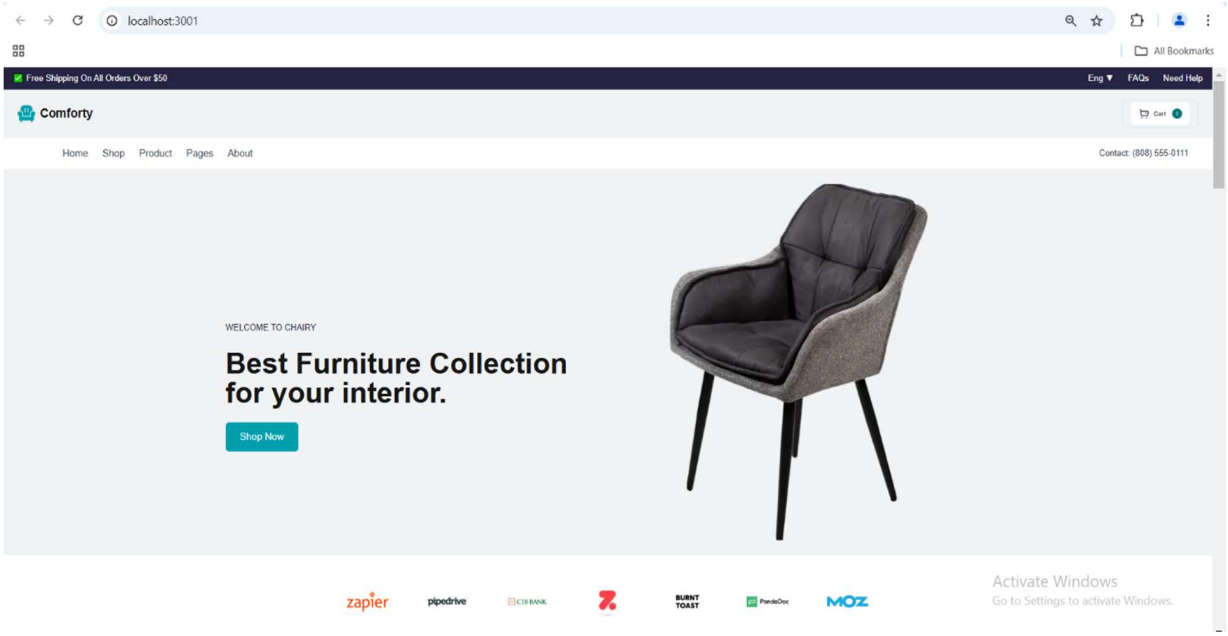
Hackathon Day 3

API Integration and Data Migration

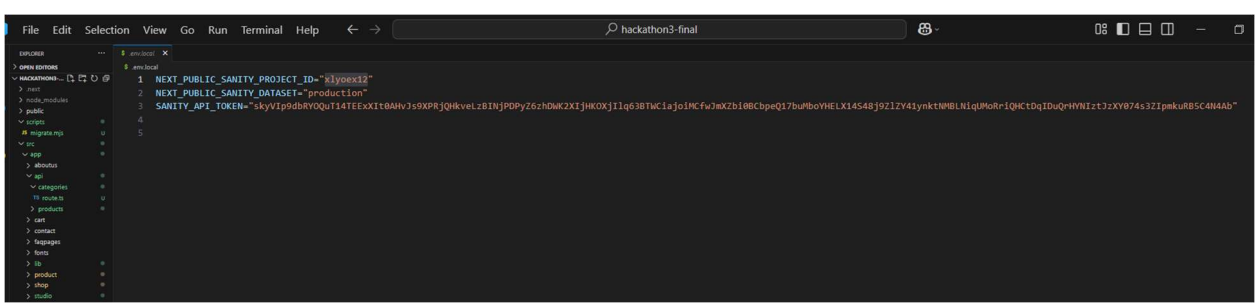


Understanding the Requirements:

The goal was to integrate a REST API and migrate its data, specifically products and categories, into a Sanity CMS using Node.js. This process also involved setting up the environment to facilitate smooth integration.

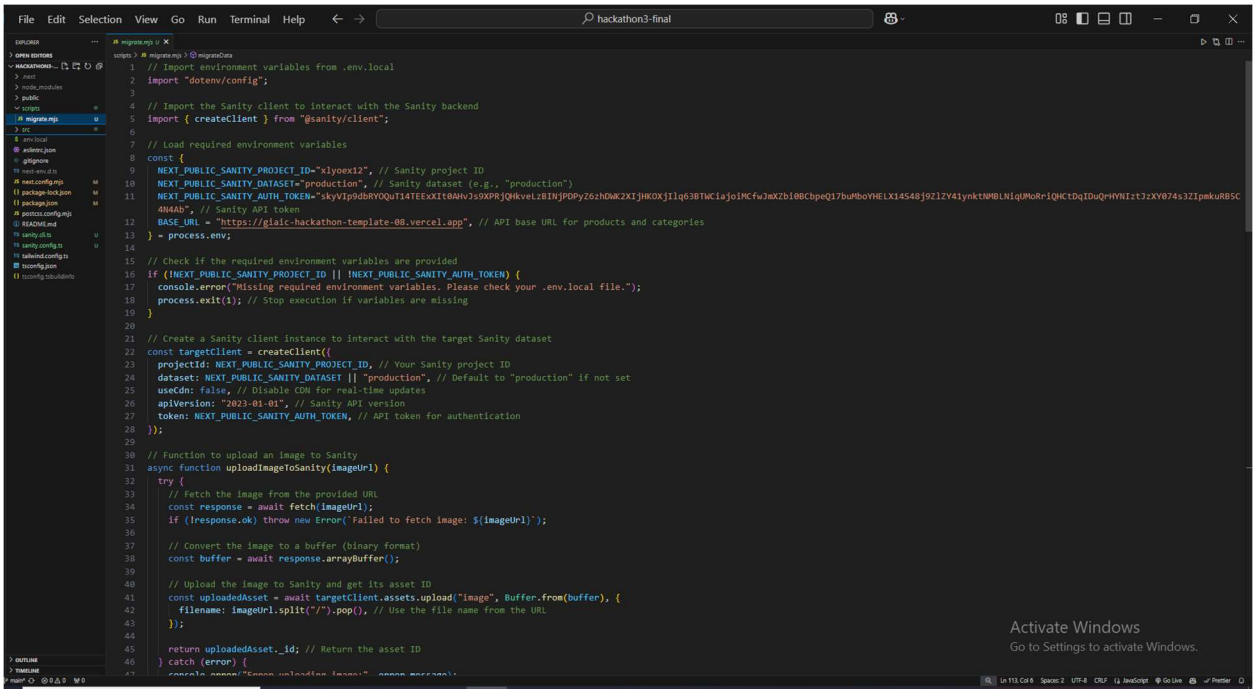
Setting Up the Environment:

- Configured `.env.local` file with the necessary Sanity credentials:
  - `SANITY_PROJECT_ID`
  - `SANITY_DATASET`
  - `SANITY_AUTH_TOKEN`
- Ensured that the correct environment variables were loaded in the script for seamless execution.

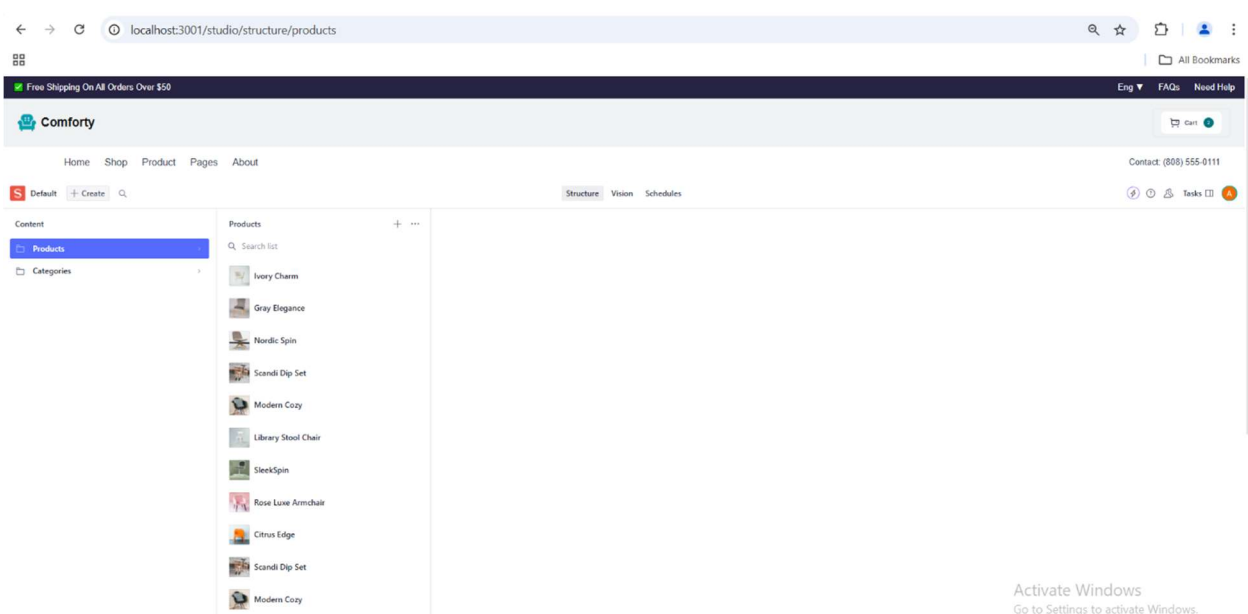


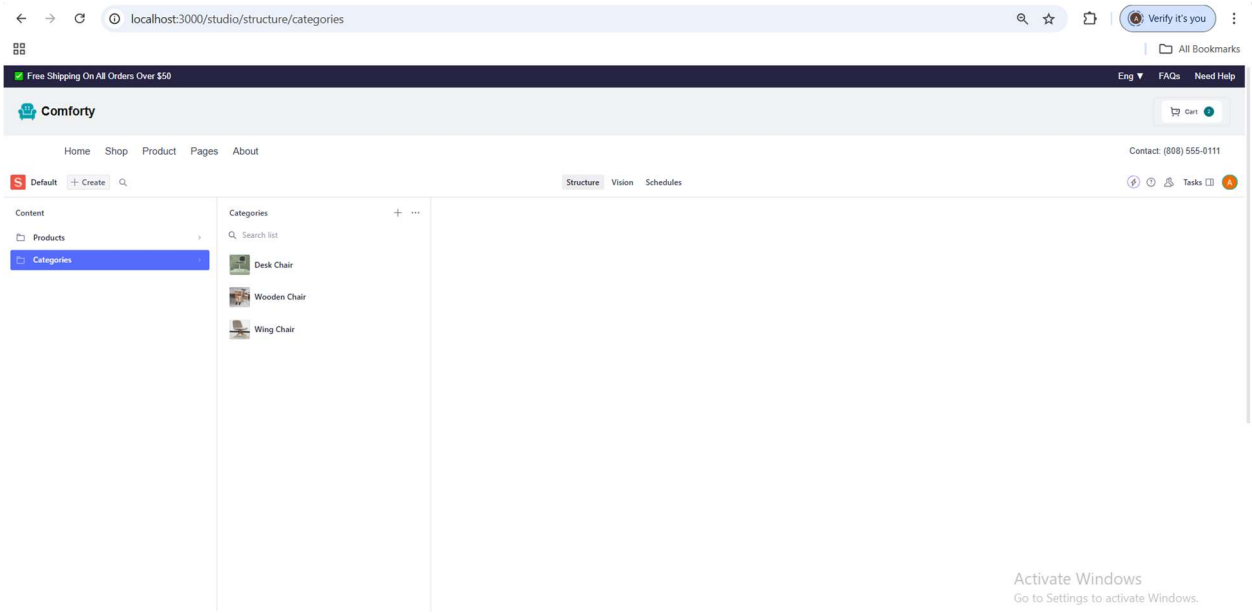
Data Migration Script:

- Developed a migration script (`migrate.mjs`) to perform the following tasks:
  - Fetch product and category data from the REST API.
  - Utilize the Sanity client to:
    - Upload images.
    - Create new records for categories and products in the Sanity CMS.



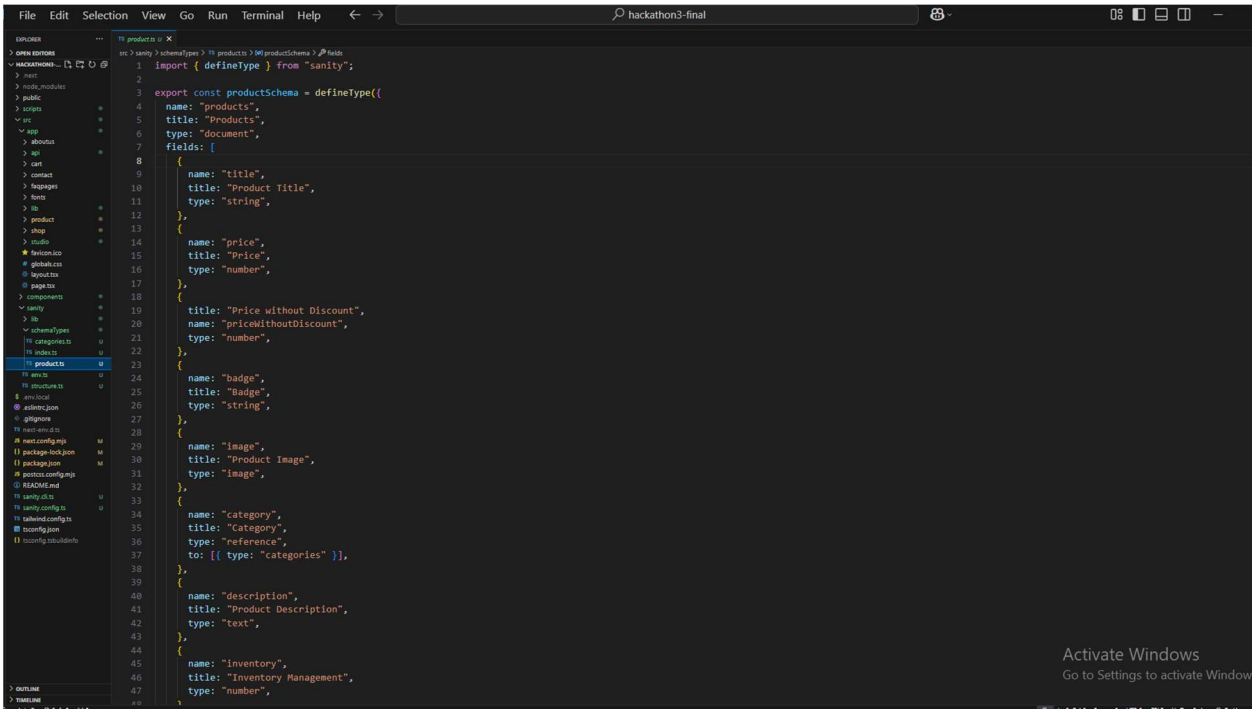
```
// Import environment variables from .env.local
import "dotenv/config";

// Import the Sanity client to interact with the Sanity backend
import { createClient } from "@sanity/client";

// Load required environment variables
const {
  NEXT_PUBLIC_SANITY_PROJECT_ID="x1yoex12", // Sanity project ID
  NEXT_PUBLIC_SANITY_DATASET="production", // Sanity dataset (e.g., "production")
  NEXT_PUBLIC_SANITY_AUTH_TOKEN="skyVIp9dbRYOQuT14TEExXIt0AHvJs9XPRjQHkvelzBINjPDPy26zhDWK2XIjHKOXjIlq63BTWCiajoiMCfwJmXZbi0BCbpeQ17buMboYHELX14S48j9ZlZY41ynktNMBLNiqUMoRriQHCtDqIDuQrHYNIztJzXY074s3ZIpmkuRB5C
  4N4Ab", // Sanity API token
  BASE_URL = "https://giaic-hackathon-template-08.vercel.app", // API base URL for products and categories
} = process.env;

// Check if the required environment variables are provided
if (!NEXT_PUBLIC_SANITY_PROJECT_ID || !NEXT_PUBLIC_SANITY_AUTH_TOKEN) {
  console.error("Missing required environment variables. Please check your .env.local file.");
  process.exit(1); // Stop execution if variables are missing
}

// Create a Sanity client instance to interact with the target Sanity dataset
const targetClient = createClient({
  projectId: NEXT_PUBLIC_SANITY_PROJECT_ID, // Your Sanity project ID
  dataset: NEXT_PUBLIC_SANITY_DATASET || "production", // Default to "production" if not set
  useCdn: false, // Disable CDN for real-time updates
  apiVersion: "2023-01-01", // Sanity API version
  token: NEXT_PUBLIC_SANITY_AUTH_TOKEN, // API token for authentication
});

// Function to upload an image to Sanity
async function uploadImageToSanity(imageUrl) {
  try {
    // Fetch the image from the provided URL
    const response = await fetch(imageUrl);
    if (!response.ok) throw new Error(`Failed to fetch image: ${imageUrl}`);

    // Convert the image to a buffer (binary format)
    const buffer = await response.arrayBuffer();

    // Upload the image to Sanity and get its asset ID
    const uploadedAsset = await targetClient.assets.upload("image", Buffer.from(buffer), {
      filename: imageUrl.split("/").pop(), // Use the file name from the URL
    });

    return uploadedAsset._id; // Return the asset ID
  } catch (error) {
```





Sanity Schema:

- **Category Schema:**
  - Fields for category name and description.
  - Reference field for linking products to categories.
- **Product Schema:**
  - Fields for product name, description, price & badge.
  - Image field to store product images.
  - Reference field to associate products with categories.



```typescript
import { defineType } from "sanity";

export const categorySchema = defineType({
    name: 'categories',
    title: 'Categories',
    type: 'document',
    fields: [
        {
            name: 'title',
            title: 'Category Title',
            type: 'string',
        },
        {
            name: 'image',
            title: 'Category Image',
            type: 'image',
        },
        {
            title: 'Number of Products',
            name: 'products',
            type: 'number',
        }
    ],
});
```

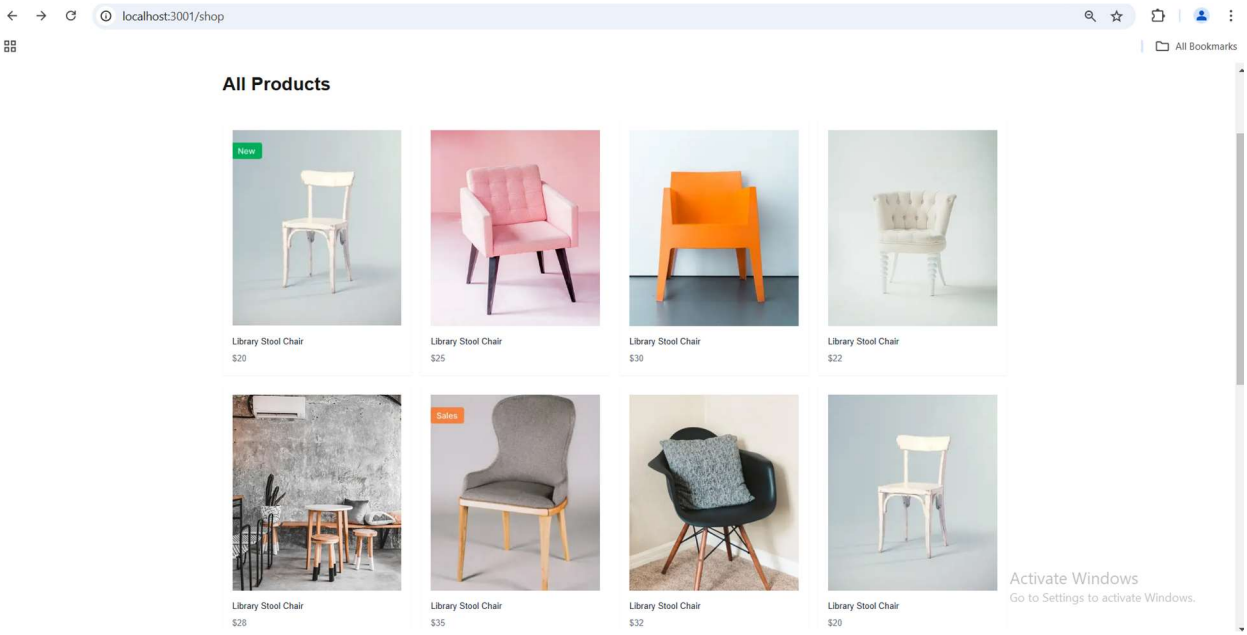
Data Migration Process:

1. Migrated categories and products to Sanity, ensuring proper data mapping.
2. Managed category mapping effectively to maintain data consistency.

```typescript
import { client } from "@/sanity/lib/client";

export const dynamic = "force-static";

export async function GET(req: { url: string}) {
  try {
    const { searchParams } = new URL(req.url);
    const slug = searchParams.get("slug");

    if (slug) {
      // Fetch a single product by slug
      const product = await client.fetch(
        `*[_type == "products" && _id == $slug][0] {
          _id,
          title,
          price,
          priceWithoutDiscount,
          badge,
          "imageUrl": image.asset->url,
          category-> {
            _id,
            title
          },
          description,
          inventory,
          tags
        }`,
        { slug }
      );

      return new Response(JSON.stringify(product), {
        headers: { "Content-Type": "application/json" },
      });
    }

    // Fetch all products
    const products = await client.fetch(
      `*[_type == "products"] {
        _id,
        title,
        price,
        priceWithoutDiscount,
        badge,
        "imageUrl": image.asset->url,
        category-> {
          _id,
```

Activate Windows
Go to Settings to activate Windows.

```typescript
import { client } from "@/sanity/lib/client";
export const dynamic = 'force-static'

export async function GET() {
  try {
    const categories = await client.fetch(`*[_type == "categories"] {
          _id,
          title,
          "imageUrl": image.asset->url
        }`);

    return new Response(JSON.stringify(categories), {
      headers: { "Content-Type": "application/json" },
    });
  } catch (error) {
    console.log(error)
    return new Response(
      JSON.stringify({ error: "Failed to fetch categories" }),
      {
        status: 500,
        headers: { "Content-Type": "application/json" },
      }
    );
  }
}
```

Finalization:

- Successfully completed the data migration process with no errors.
- Confirmed that the project is ready for front-end integration.